Adaptive Path-Planning for Autonomous Robots: A UCH-Enhanced Q-Learning Approach

Wei Liu, *Member, IEEE,*Ruiyang Wang,Haonan Wang and Guangwei Liu

***Abstract*—Q-learning method can address robot path planning issues, but it still faces challenges in search and update efficiency. We propose an enhanced Ant Colony Optimization algorithm to optimize Q-table initialization and use the UCH mechanism to refine the reward function. Extensive experiments were conducted in three different raster grid environments, validating the accuracy of our method and demonstrating its superior path-planning capabilities compared to traditional methods.**

***Index Terms*—Path Planning, IQL algorithms, UCH mechanism, Chebyshev distance.**

I. Introduction

WITH the rapid development of AI and control technology, mobile robots have been widely used in industrial manufacturing, logistics and sorting, etc. Intelligent control of mobile robots is evolving towards self-learning and adaptation. In recent years, autonomous mobile robots have gained significance due to their relevance to contemporary societal needs [1]. They are among the fastest-growing fields, capable of moving autonomously in various environments without external assistance [2]. The movement of intelligent mobile robots in complex environments is a key research interest [3]. Motion planning is a critical task for intelligent mobile robots, divided into path planning and trajectory planning [4]. In practical applications, path planning often serves as the initial step of trajectory planning. Its primary goal is to find the optimal path from a starting point to an endpoint in a given environment, with wide applications in driverless vehicles, logistics, and robot navigation. The environment in which a mobile robot operates is crucial for path planning. It can be static and unchanged or dynamically transforming. Path planning in dynamic environments is more practical and challenging [5].

In recent years, various path planning algorithms have been applied to mobile robots, including Dijkstra's, Bellman-Ford, A-Star, Rapidly-exploring Random Tree (RRT), Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO), Genetic Algorithm (GA), Artificial Potential Field (APF), and Reinforcement Learning (RL). In [6], an improved Dijkstra algorithm is proposed for path optimization, combining the octagonal search method with the Dijkstra algorithm. In [7], a Bellman-Ford algorithm is proposed to solve the shortest path problem in a network within a fuzzy image environment. In [8], a path planning method based on geometric A-Star algorithm is proposed and applied in various scenarios. In [9], a convolutional neural network-based RRT algorithm is proposed for solving path planning problems. In [10], an improved Ant Colony Optimized Artificial Potential Field (ACO-APF) algorithm is proposed, combining the ACO algorithm with the artificial potential field method. This lattice graph-based algorithm is used for local and global path planning of unmanned underwater vehicles (UUVs) in dynamic environments. In [11], a new strategy for planning smooth paths for mobile robots is proposed, based on particle swarm algorithms with successive higher-order Bessel curves. In [12], an improved Genetic Algorithm (GA) is proposed to enhance mobile robots' ability to solve path planning problems in complex maps. Basic path planning algorithms, while capable of solving issues like shortest path search, still face significant challenges. These include difficulties in environment modeling, adapting to complex environments, slow algorithm convergence, and a tendency to fall into local optima.

To tackle the challenges traditional algorithms face in modeling unknown environments and enhancing convergence speed, researchers have employed reinforcement learning (RL) algorithms for path planning. In [13], the G2RL approach is introduced to solve the path planning problem in large dynamic environments. In [14], the MAPPER approach is proposed, a decentralized, partially observable evolutionary reinforcement learning method for hybrid dynamic environments. In [15], deep neural networks and Markov Decision Process (MDP) are used for autonomous navigation of small UAVs. In [16], DRL-based algorithms are used alongside structured map data to train DDQN architectures, balancing navigation with mission objectives. In [17], the SAC algorithm combined with deep reinforcement learning is used to achieve dynamic obstacle avoidance for a robotic arm. In [18], algorithms based on two-depth Q-networks are proposed to develop QoS-based action selection strategies. In [19], an optimized TD3 model is proposed for energy-efficient and effective path planning for UAVs. In [20], a deep reinforcement learning strategy is proposed to enhance USV autopilot and collision avoidance capabilities by interacting with a simulation environment

Manuscript received xxxx, revised xxxx; accepted xxxx.

This work was supported in part by the National Natural Science Foundation of China under Grant 52374123 and Grant 51974144, in part by the Project of Liaoning Provincial Department of Education under Grant LJKZ0340, and in part by a Project supported by Liaoning Revitalization Talents Program under Grant XLYC2211085. (*Corresponding author*: *Wei Liu*).

Wei Liu is with the College of Science, Institute of Mathematics and Systems Science and Institute of Intelligent Engineering and Mathematics, Liaoning Technical University, Fuxin 123000, Liaoning, China. (e-mail: liuwei@lntu.edu.cn)

Ruiyang Wang is with the College of Science, Liaoning Technical University, Fuxin 123000, Liaoning, China. (e-mail: 472321492@stu.lntu.edu.cn)

Haonan Wang is with the Johns Hopkins University, Baltimore 21218, Maryland, USA.(e-mail: hwang298@jh.edu)

Guangwei Liu is with the College of Mines, Liaoning Technical University, Fuxin 123000, Liaoning, China.(e-mail: liuguangwei@lntu.edu.cn)

through DQN. In [21], a USV collision avoidance algorithm based on COLREGs is proposed, using a DDPG network and cumulative priority sampling to enhance efficiency. In [22], the MADPG method and Gumbel-Softmax strategy are used for AGV conflict prevention path planning. In [23], deep reinforcement learning methods are proposed for driverless ground vehicles to enable real-time planning in harsh environments. Although RL is effective in local path planning, it still faces challenges such as dimensional catastrophe and slow convergence speed.

Researchers have widely adopted Q-learning and its enhanced algorithms to address path planning in reinforcement learning, achieving remarkable results. In [24], DFQL algorithm combines Q-learning with artificial potential fields to effectively solve the path planning problem for unmanned underwater vehicles in partially known marine environments. In [25], the EQL algorithm provides fast convergence to the optimal path for mobile robots through innovative reward functions and selection strategies, improving path optimization, computational efficiency, and safety. However, in [26], the initialization of the Q-table is crucial to the performance of the algorithm. Hao Bing et al., in [27], optimized the Q-table initialization using the Flower Pollination Algorithm (FPA) to improve path search efficiency. Although heuristic algorithms [28], [29] provide feasible solutions for complex problems, there are non-optimal solutions and complexity analysis difficulties, which need to be further improved to enhance the algorithm efficiency and the learning speed of intelligences. In addition, Q-learning algorithm also suffers from credit allocation difficulties, falling into local optima, the curse of dimensionality, and over-optimization of the reward function [30], [31]. Difficulty in credit allocation can cause time delays, making it hard to assess actions in real-time [32]. Local optima can prevent agents from finding better strategies [33]. Dimensionality issues can degrade model performance and increase computational complexity [34]. Over-optimization may lead to poor generalization in new environments or tasks. To address the above problems, researchers have proposed a variety of improved reward function mechanisms [35]. For example, John von Neumann and Oskar Morgenstern, pioneers in the field of reinforcement learning, introduced utility theory in game theory, which was later used to design more complex reward functions [36]. Some scholars have also proposed a variety of methods to improve reward function design, using hierarchical reinforcement learning and potentially model-based approaches to improve learning efficiency and generalisation [37]. Meiyan Zhang et al [38] introduced a novel reward function based on the Predator-Prey model to improve algorithm performance. Although these studies advance the reward function design, the adaptability and generalization ability of mobile robot path planning in dynamic environments still need further research and improvement [39].

In this work, an improved Q-learning algorithm with UCH mechanism and optimized Q-table initialization is proposed to enhance path planning efficiency and selection accuracy. The contributions of this paper are:

1) A variant is introduced that enhances performance by dynamically adjusting pheromone volatility, thus avoiding local optima.
2) We improve the Q-table initialization process of the Q-learning algorithm using an enhanced ACO algorithm.
3) A UCH reward mechanism is introduced that dynamically adjusts reward parameters, reducing exploration dilemmas and enabling more precise reward criterion assessment, thereby enhancing algorithm performance.

The subsequent sections of this document are structured as follows: Section 2 details the basics of RL, traditional Q-learning algorithms, and environmental modeling for path planning. Section 3 outlines an upgraded Ant Colony Optimization (PACO) algorithm to refine Q-table initialization, describes the UCH mechanism for tuning the reward function, and introduces the IQL algorithm along with its evaluation metrics. Section 4 elaborates on the simulation methodologies and experimental outcomes. Finally, Section 5 encapsulates the paper's principal contributions.

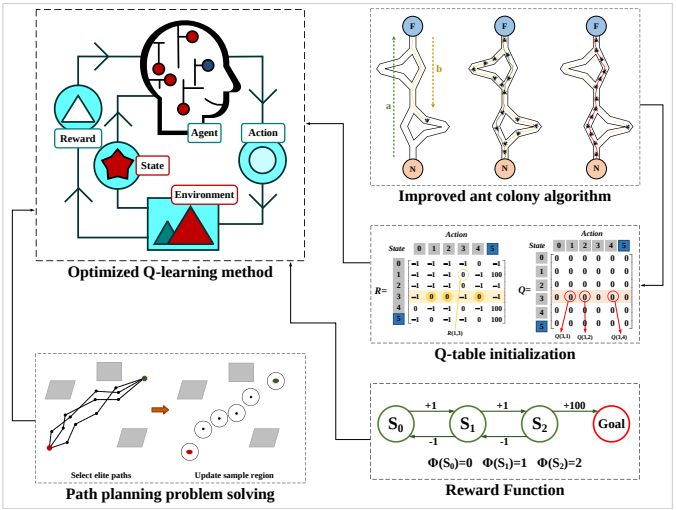

Fig. 1. Framework diagram of the improved Q-learning (IQL) algorithm

II. Background

This section covers the basics of RL and traditional Q-learning, focusing on reward functions and environment modeling essential for the improved Q-learning (IQL) algorithm proposed in this paper.

A. Reinforcement Learning

In the standard RL model, its fundamental framework is composed of three core elements: state, action, and reward. Agent [40] acquires the current state s_t of the environment through observation at moment t, and then performs a specific behavior a from all feasible behaviors according to an established strategy and receives a reward r_t for evaluating the merit of the behavior. Subsequently, the intelligent body transfers to a new state s_{t+1} based on the performed behavior and continues the process until the training is complete. The ultimate goal of the intelligent body is to learn a strategy $S \rightarrow A$ that results in maximizing the desired cumulative reward r. In this process, the set of states of the environment

is denoted as $S = \{s_1, s_2, \ldots, s_{t+1}, \ldots\}$, while all the possible behaviors constitute the set $A = \{a_1, a_2, \ldots, a_n\}$. Each behavior has a corresponding Q-value by which the intelligent body decides its behavioral choices.

B. Basic Q-learning algorithms

Q-learning algorithm [41], introduced in 1989, is a model-free reinforcement learning method that guides an agent's actions across various states. As a classic algorithm, it doesn't require model construction and consists of three main components [42]: Q-table initialization, action selection strategy, and Q-table update. Typically, the Q-table is initialized with constant values, and the ϵ-greedy strategy is used for action selection. The updated formula for the value function is shown in equation (1).

$$Q(s,a) = Q(s,a) + \alpha\left(r(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a)\right) \tag{1}$$

In addition to the three essential components, defining the reward function and collision handling is crucial for the path planning problem. The agent receives a penalty for each step taken, which is related to the length of the path from the previous step. When the agent hits an obstacle, it remains in place and seeks actions to avoid the obstacle until reaching the endpoint. The reward function can be defined as shown in equation (2).

$$r(s,a) = -\sqrt{(x_s - x_s'')^2 + (y_s - y_s'')^2} \tag{2}$$

where s is the current agent state, and x_z, y_z is the coordinates of the current state corresponding to the raster; s'' is the previous state of the agent, and (x_z'', y_z'') is the coordinates of the raster corresponding to the previous state.

Basic flow of Q-learning algorithm is depicted in Fig. 2.

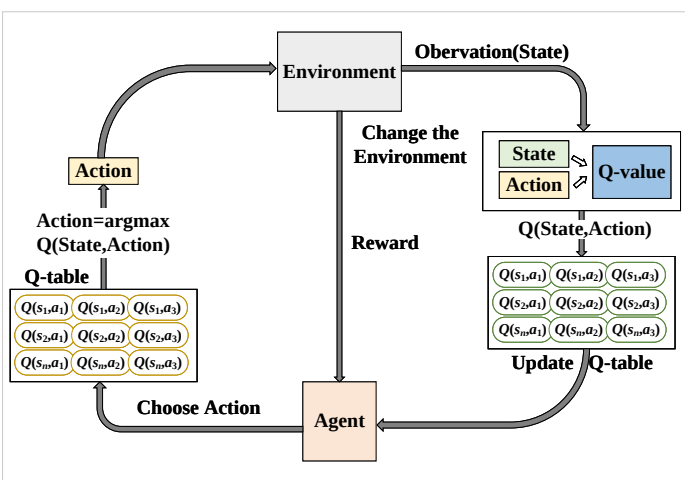

Fig. 2. Interplay process of the Q-learning algorithm and the surrounding environment

Q-learning is widely used in gaming, robotics, and resource management due to its simplicity and effectiveness, providing a solid foundation for reinforcement learning. However, traditional Q-learning faces challenges such as slow iteration speed, difficulty in environment comprehension during Q-table initialization, and a tendency to fall into local optima.

C. Environmental modelling for path planning

This paper uses the raster method to model the environment, which includes static obstacles [43]. A grid cell that contains an obstacle is marked as "1" and represented by a black grid in the simulation drawing, regardless of whether the obstacle completely covers the cell. Conversely, a grid cell that has no obstacle is marked as "0" and represented by a white grid in the simulation drawing.

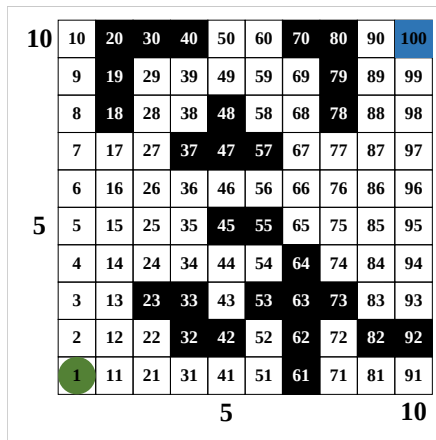

Fig. 3. Raster map environment and serial number encoding

Fig. 3 is used as an example to illustrate the relationship between raster coding and horizontal and vertical coordinates [44]. Firstly, a Cartesian coordinate method is used to number grids: horizontal number of grid is used as the abscissa of grid, and vertical number of grid is used as the ordinate of grid. In Fig. 3, the maximum h of the raster abscissa is 10, the maximum v of the raster ordinate is 10, and the total number of rasters is 100. Grids are numbered $1, 2, \cdots, h \times v$ from bottom to top and left to right. The relationship between the grid coordinates and the grid number is expressed as the following formula. Table I shows the symbols and their meanings in (3).

$$\begin{cases} x_c = ceil(c/h); \\ y_c = \begin{cases} \mathrm{mod}(c/h), \mathrm{mod}(c/h) \neq 0; \\ h \qquad\qquad , \mathrm{mod}(c/h) = 0. \end{cases} \end{cases} \tag{3}$$

TABLE I
EXPLANATION OF THE FORMULA SYMBOLS FOR THE RELATIONSHIP BETWEEN GRID COORDINATES AND GRID NUMBERS

Symbols	Symbolic meanings
x_c	Grid abscissa
y_c	Grid ordinates
ceil(x)	Take an integer operation that is not less than x
mod(x,y)	Take the remainder of x/y
c	The current number of the raster

Path planning algorithm, which adopts an obstacle avoidance strategy, ensures that the planned path neither passes through obstacles nor collides with them. Fig. 4, which illustrates agent's movement from grid c to grid n, shows the path options in 8 directions and the correct detour path.

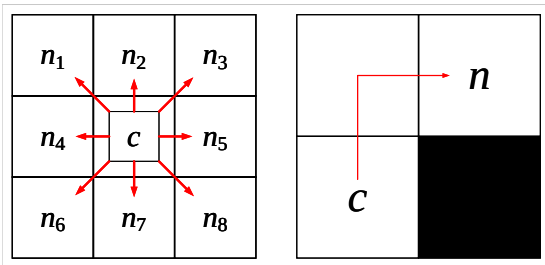

Fig. 4. The selected path and the correct detour path

Fig. 5 illustrates two common error paths, which collide with obstacles and do not meet basic requirements of conditional path constraints.

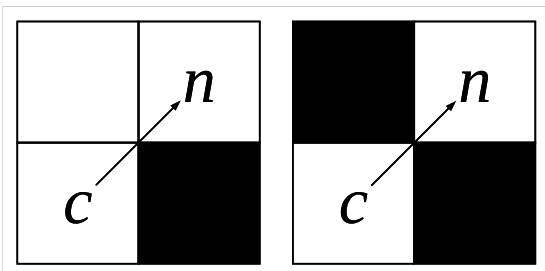

Fig. 5. Two error paths

III. Methods

The IQL algorithm introduced in this paper enhances Q-learning for path planning by addressing the inefficiencies of traditional Q-tables. Leveraging the PACO algorithm for Q-table initialization, IQL enables quicker adaptation and more efficient path selection, improving both solution efficiency and accuracy. ACO was chosen over alternatives like Genetic Algorithms and Particle Swarm Optimization for its superior global optimization, adaptability to dynamic environments, and balanced exploration-exploitation. Additionally, ACO's effectiveness in combinatorial optimization and scalability make it ideal for complex path-planning tasks.

Furthermore, the UCH mechanism is implemented to handle sparse reward functions, reducing exploration risks and enhancing the agent's ability to identify reward criteria. These combined optimizations significantly enhance the IQL algorithm's performance, leading to faster convergence, higher path quality, and improved obstacle avoidance, making it the preferred choice for efficient and adaptable path planning.

Overall, the IQL algorithm, which effectively adapts to the comprehensive needs of path planning efficiency, accuracy, and adaptability, accelerates convergence speed, improves path quality, and strengthens obstacle avoidance.

A. Q table initialization optimization strategy

1) ***PACO algorithm****:* Ant colony optimization (ACO) [45] simulates the foraging behavior of real ants. During foraging, ants discharge a volatile substance called pheromones along their path, using its presence and quantity to guide their movement direction. Generally, ants tend to choose paths with more pheromones, forming a positive feedback mechanism: the pheromones on the optimal path increase, while those on other paths gradually decay over time. The ACO algorithm, which simulates this foraging action [46], is a heuristic optimization algorithm with certain advantages in solving combinatorial optimization problems. However, it also has disadvantages such as slow convergence speed, sensitive parameter settings, and a tendency to fall into local optima, which can affect the algorithm's performance and applicability.

To address these issues, this paper introduces the PACO algorithm to improve ACO algorithm performance. The PACO algorithm is used as the optimization strategy for Q-table initialization, where the best solution obtained by the ACO algorithm serves as the Q-table initialization strategy.

The PACO algorithm draws on the feeding behavior characteristics of ants in nature to form a distributed intelligence system, and the pheromone update mechanism is improved by introducing the volatilization coefficient of the pheromone to avoid the problem of ants falling into a local optimum, as shown in Fig. 6.

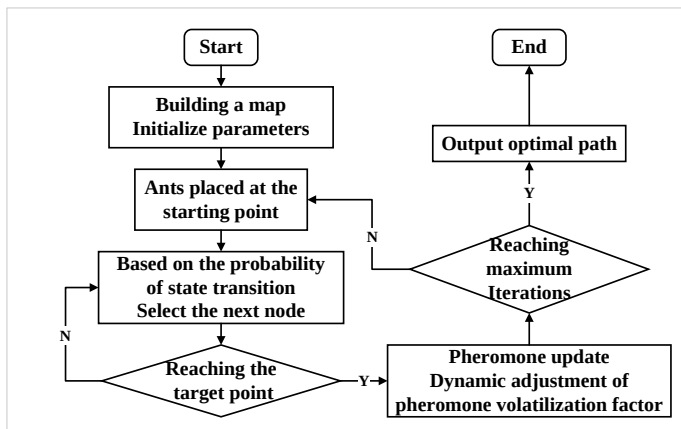

Fig. 6. Flow chart of the PACO algorithm

PACO algorithm model is briefly described with the help of n random regions. Let m ants be placed on n random regions, where n is the size of the aggregation point; m denotes ants' number; c is a set of random areas of this problem; $\tau_{ij}(t)$ is the pheromone on the path region of the random region i and j at time t.

Step1: State Transition Guidelines.

Each ant independently selects the next random region to transfer to based on the pheromones in each path and records the random region that ant k has traveled in the $tabu_k$ table. At time t, the likelihood of ant k moving from random region i to random region j, denoted as $p_{ij}^k(t)$, is as (4).

$$p_{ij}^k(t) = \begin{cases} allowed_k = \{C - tabu_k\} & \\ \frac{[\tau_{ij}(t)]^\alpha g[\eta_{ik}(t)]^\beta}{\sum\limits_{s \in allowed_k} [\tau_{is}(t)]^\alpha g[\eta_{is}(t)]^\beta} & j \in allowed_k \\ 0 & j \notin allowed_k \end{cases} \tag{4}$$

Among them, α is the information heuristic, reflecting the influence of pheromones on the path region of the ant, and β is the expectation heuristic, indicating the impact of the path area on the ant. $allowed_k$ is the random region set that can be selected when the ant k moves next.

Step2: Pheromone updates.

In the PACO algorithm, the probability of choosing a path is proportional to the pheromone concentration on that path. Over time, pheromone levels decrease according to a defined volatility coefficient. Traditional ant colony algorithms use a fixed value for this coefficient. If set incorrectly, the colony may prematurely converge on certain paths, resulting in local optima, thereby hindering the algorithm's ability to find the global best solution and slowing down convergence.

To address this limitation, the PACO algorithm incorporates a mechanism for dynamically adjusting the pheromone volatility factor. Instead of being a static constant, the pheromone volatility factor ρ decreases gradually with each iteration. This adjustment maintains path diversity and enhances convergence speed, as demonstrated in (5).

$$\rho(t) = \frac{\lambda_1}{(1 + e^{\frac{\lambda t}{3m}})} \tag{5}$$

where λ_1 represents the adjustment coefficient of the pheromone volatilization factor, *Nc* represents the number of the current iteration, and *m* is the ant's number.

The pheromone update formula in the PACO algorithm is shown in (6).

$$\begin{cases} \tau_{ij}(t+1) = (1-\rho(t)) \bullet \tau_{ij}(t) + \Delta\tau_{ij}(t) \\ \Delta\tau_{ij}(t) = \sum\limits_{k=1}^{m} \Delta\tau_{ij}^{k}(t), \Delta\tau_{ij}^{k}(t) = \begin{cases} \frac{Q}{L_k}, (i,j) \in L_k \\ 0 \quad , others \end{cases} \end{cases} \tag{6}$$

Table II shows the symbols and their meanings in (6).

TABLE II
SYMBOLS OF PHEROMONE UPDATE FORMULAS IN THE PACO ALGORITHM

Symbols	Symbolic meanings
$\Delta\tau_{ij}(t)$	The pheromone increment on path (*i*,*j*) at time *t*
k	The *k*-th ant
m	Total ants number
Q	Initial intensity of pheromone increment (fixed constant)
L_k	The *k*-th ant searches for the total length of the path at time *t*

*2) **Q-table initialization operation**:* Q-table initialization optimization strategy basis of the PACO algorithm involves these steps, which could be summarized to describe the process. Fig. 7 shows the Q-table initialization optimization strategy process based on the PACO algorithm.

Step1: Problem definition. Identify the problem of finding the optimal path that needs to be solved.

Step2: PACO algorithm parameter settings. Set the parameters of the PACO algorithm, such as the total number of ants, the initial intensity of pheromone increment, the expected heuristic, and the pheromone volatilization rate.

Step3: Colony initialization. Randomly place ants at the start of the map, with each ant representing a potential solution.

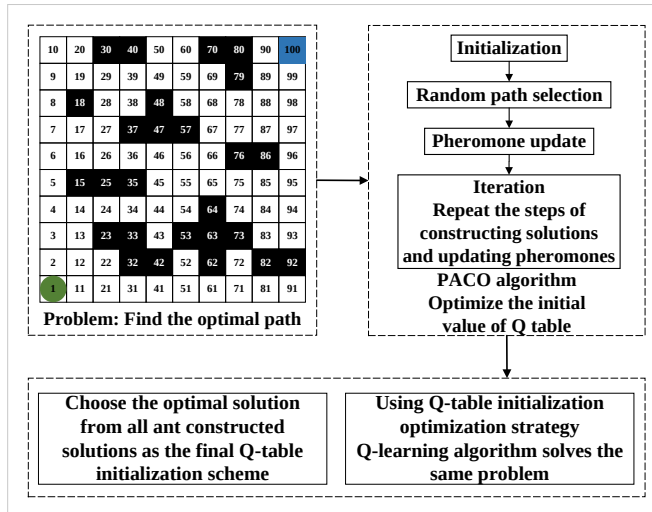

Fig. 7. Initialization optimization policy process of the Q-table

Step4: Iterative process. Select the next node of the path based on the state transition probability and determine whether the target point is reached.

Step5: Pheromone updates. The path diversity is ensured by introducing the mechanism of dynamic regulation of pheromone volatilization factors, and the convergence rate is improved.

Step6: Output the result. After the algorithm is terminated, the best solution found is output as the initial value of the Q table.

Step7: Q-table initialization. The initial utility value of the ant taking different actions in each state is the IQL algorithm's initial value. This step is a vital part of the optimization strategy.

Step8: Algorithm performance evaluation. The results are evaluated to evidence the effectiveness of the optimization strategy.

B. Reward function optimization

The reward function, a crucial component of Q-learning algorithms, determines the reward an agent receives from interacting with the environment. By optimizing the reward function, the agent can learn an effective strategy more quickly. In this section, we optimize the reward function by introducing the UCH mechanism, which significantly enhances the algorithm's performance.

The UCH mechanism dynamically adjusts the reward parameter and changes the distance calculation method. This mechanism helps the reward function develop an effective strategy faster and in real-time. Table III uses Euclidean distance as an example to compare the optimized and original reward functions.

In this paper, we compare two distance calculation methods with the Euclidean distance method used in the original algorithm.

(1) Chebyshev distance

In mathematics, Chebyshev distance [47], also known as the $L\infty$ norm, is regarded as a measure within a vector space. The

TABLE III
COMPARISON BETWEEN THE ORIGINAL REWARD FUNCTION AND THE OPTIMIZED REWARD FUNCTION (DISTANCE CALCULATION METHOD: EUCLIDEAN DISTANCE)

Meaning	Formula
Raw reward function	$r(s,a) = -\sqrt{(x_s - x_s'')^2 + (y_s - y_s'')^2}$
Parameter formulas	$\mu(t) = \mu_0 \cdot \frac{1}{\pi + \pi \cdot e^{-t}}$
Optimized reward function	$r^O(s,a) = -\mu(t)\, r(s,a)$

distance is determined by calculating the absolute difference between two points in each coordinate dimension and taking the maximum value from these differences.

Chebyshev distance is based on the concept of a consistent norm (or supremum norm) and is classified as an injective metric space.

The scope of this paper is the influence of distance calculation methods on the performance of the algorithm on the two-dimensional plane, so only the Chebyshev distance on the two-dimensional plane is introduced. Let the Chebyshev distance between the two points on the plane be $A(x_s, y_s)$ and $B(x_s'', y_s'')$, and the Chebyshev distance of the two points *AB* as (7).

$$d_{AB}^{Q} = \max(|x_s - x_s''|, |y_s - y_s''|) \tag{7}$$

Table IV compares the optimized reward function with the original reward function using the Chebyshev distance as an example.

TABLE IV
COMPARISON BETWEEN THE ORIGINAL REWARD FUNCTION AND THE OPTIMIZED REWARD FUNCTION (DISTANCE CALCULATION METHOD: CHEBYSHEV DISTANCE)

Meaning	Formula
Raw reward function	$r_q(s,a) = -\max(\lvert x_s - x_s''\rvert, \lvert y_s - y_s''\rvert)$
Parameter formulas	$\mu(t) = \mu_0 \cdot \frac{1}{\pi + \pi \cdot e^{-t}}$
Optimized reward function	$r^Q(s,a) = \rho(t) \cdot\ r_q(s,a)$

(2) Manhattan distance

In Manhattan neighborhoods, the driving distance from one intersection to another is not a straight-line distance between two points, and this actual distance is the "Manhattan distance [48]". For this reason, Manhattan distance is also known as "taxi distance" or "city block distance".

The scope of this paper is the impact of distance calculation methods on the performance of the algorithm on the two-dimensional plane, so only the Manhattan distance on the two-dimensional plane is introduced. Let the two points on the plane be $A(x_s, y_s)$ and $B(x_s'', y_s'')$, and the Manhattan distance between the two points *AB* as (8).

$$d_{AB}^{M} = |x_z - x_z''| + |y_z - y_z''| \tag{8}$$

Fig. 8 below illustrates the difference and connection between the Euclidean distance, the Manhattan distance and the equivalent Manhattan distance.

Table V below takes the Manhattan distance as an example to show the comparison between the optimized reward function and the original reward function.

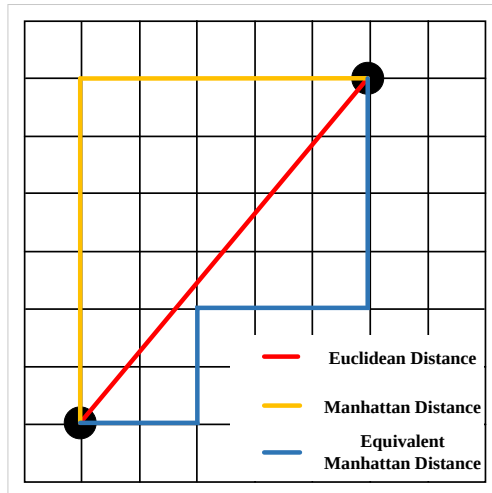

Fig. 8. Euclidean distance, Manhattan distance, and equivalent Manhattan distance

TABLE V
COMPARISON BETWEEN THE ORIGINAL REWARD FUNCTION AND THE OPTIMIZED REWARD FUNCTION (DISTANCE CALCULATION METHOD: MANHATTAN DISTANCE)

Meaning	Formula
Raw reward function	$r_m(s,a) = -\max(\lvert x_s - x_s''\rvert, \lvert y_s - y_s''\rvert)$
Parameter formulas	$\mu(t) = \mu_0 \cdot \frac{1}{\pi + \pi \cdot e^{-t}}$
Optimized reward function	$r^M(s,a) = \rho(t) \cdot\ r_m(s,a)$

C. Evaluation indicators

The algorithm evaluation index is a quantitative measure of algorithm performance, encompassing learning speed, stability, and final results. These indicators provide a comprehensive evaluation of the optimized algorithm's practical performance and form the basis for further improvement assessments.

In this study, three evaluation indicators are used to assess the improved Q-Learning (IQL) algorithms.

(1) Trials number with the same number of iterations X. With the same number of iterations, the algorithm requires fewer experiments to reach a steady state, indicating improved performance. This evaluation index $\eta(t)$ reflects the learning speed and efficiency of the algorithm.

(2) The number of trials with a standard deviation of 0. The trial number required to reach a standard deviation of 0 ($d(t)$) is a statistic that measures the degree of dispersion in a set of numerical distributions. If the optimized algorithm achieves a standard deviation of 0 with fewer tests, it indicates improved measurement stability, reduced systematic error, and enhanced data reliability.

(3) The expected return after the algorithm's convergence. Expected return ($e(t)$) refers to the average cumulative return an agent receives from the initial state when following a strategy. The agent can better utilize the learned strategy to maximize cumulative rewards if the algorithm provides a higher expected return in the initial state.

Therefore, the formula $J(t)$ that defines the performance evaluation index of the experiment in this paper is as (9).

$$J(t) = \frac{\frac{\eta(1)-\eta(2)}{\eta(1)} + \frac{d(1)-d(2)}{d(1)} + \frac{e(1)-e(2)}{e(2)}}{3} \times 100\% \quad (9)$$

Table VI illustrates the meaning of the symbols in (9). The i's in the table VI are 1 or 2.

TABLE VI
DESCRIPTION OF THE SYMBOL OF $J(t)$

Symbols	Symbolic meaning
1	The original algorithm
2	The improved algorithm
$\eta(i)$	Trials number for the algorithm to reach steady state
$d(i)$	The algorithm's trials number with standard deviation to 0
$e(i)$	The algorithm accumulates expected returns

IV. EXPERIMENTS

A. Simulation environment

The algorithm runtime environment and computer configuration in this document are shown in Table VII.

TABLE VII
THE ALGORITHM RUNTIME ENVIRONMENT AND COMPUTER CONFIGURATION

Parameter	Configuration
Device name	LAPTOP-A3S8EAVD
Processor	Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
With RAM	8.00 GB (7.85 GB available)
System type	64-bit operating system, x64-based processor
Simulation software	MATLAB R2022a

Three MATLAB-designed raster map environments of varying complexity were implemented to evaluate the IQL algorithm's performance in route planning. These environments simulate real-world robot path planning, with each cell representing a potential robot position. Black grids denote obstacles, while white grids represent navigable paths, providing a clear framework for algorithm development and testing.

The first raster map, shown in Fig.9 (S10), is a low-dimensional, simple environment with a few obstacles, where the robot can quickly find the optimal route between the start and endpoints. This setup is used to test the basic functionality and learning efficiency of the IQL algorithm.

The second environment, i.e., the map of Fig.10 (S20), is of medium dimensionality and has a more complex structure, including more black grids. This environment allows for the testing of the route planning ability and computational efficiency of the IQL algorithms in the face of more complex situations.

The third environment, the map in Fig.11 (S30), is a high-dimensional complex map designed with a large number of obstacles. This environment simulates a challenging navigation task in the real world. It is used to estimate the performance of the IQL algorithm in dealing with elaborate environments, especially in planning speed and obstacle avoidance.

A series of simulation experiments in three different dimensional raster map environments were conducted to compare the performance of the IQL algorithm with traditional Q-learning and other algorithms. This comprehensive evaluation

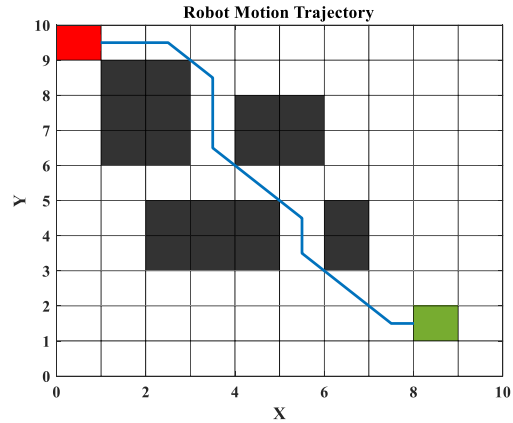

Fig. 9. 10*10 raster map environment (S10)

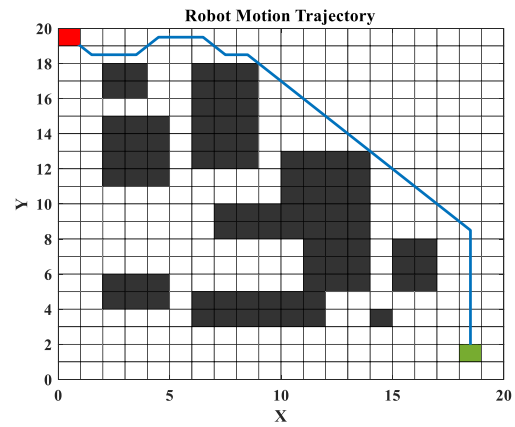

Fig. 10. 20*20 raster map environment (S20)

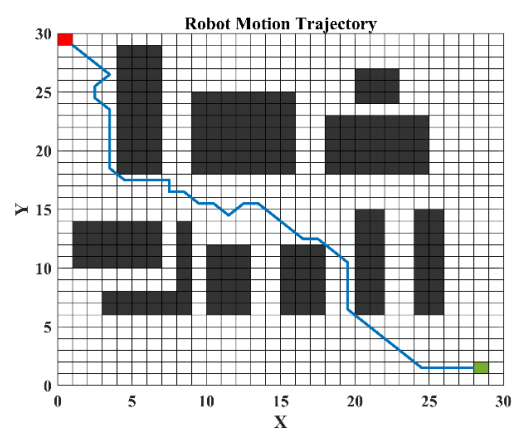

Fig. 11. 30*30 raster map environment (S30)

highlights the practical effects and potential application value of the proposed optimization methodology for path planning problems.

Basic parameter settings chosen for this experiment are presented in Table VIII.

TABLE VIII
IQL ALGORITHMS PARAMETER SETTINGS

Name of the parameters involved	Value or range
Learning rate (α)	0.3
Discount factor (γ)	0.95
Q-Table Initializations	4.5598/11.5598/9.3397/7.5598
$\rho_0$in the optimized reward function	0.156/0.0156/0.016/0.01056
Convergence target	0.25
Convergence Iterations Average	10

B. Algorithm validation

In this section, the following four algorithms are tested in each of the above three environments:

a). Q-learning algorithm.

b). A Q-learning algorithm for optimizing the initial values of Q-tables.

c). Q-learning algorithm with improved reward function.

d). IQL algorithm.

After preliminary experiments and parameter tuning, the IQL algorithm achieved optimal learning and path planning with a Q-table initialized at 9.3397, ρ_0 set to 0.016 in the optimized reward function, and Chebyshev distance for computation. These parameters were used in all subsequent experiments. The following tables (Table IX, Table X and Table XI) present the results for a Q-table initialized at 9.3397, ρ_0 set to 0.016, comparing Chebyshev and Manhattan distance calculations.

TABLE IX
EXPERIMENTAL RESULTS OF FOUR ALGORITHMS IN A 10*10 RASTER MAP ENVIRONMENT

Algorithm name	Distance calculation	$\eta(t)$	$d(t)$	$e(t)$
a)	Euclidean distance	157	147	2543.26
b)	Euclidean distance	143	133	2680.80
c)	Chebyshev distance	137	127	2674.77
c)	Manhattan distance	145	135	2666.29
d)	Chebyshev distance	133	123	2726.95
d)	Manhattan distance	142	132	2686.52

TABLE X
EXPERIMENTAL RESULTS OF FOUR ALGORITHMS IN A 20*20 RASTER MAP ENVIRONMENT

Algorithm name	Distance calculation	$\eta(t)$	$d(t)$	$e(t)$
a)	Euclidean distance	486	476	8794.61
b)	Euclidean distance	430	420	9210.92
c)	Chebyshev distance	426	416	9156.23
c)	Manhattan distance	432	422	9150.13
d)	Chebyshev distance	376	366	9594.86
d)	Manhattan distance	408	398	9395.92

Overall, the IQL algorithm performance is significantly improved for simple scenes in low dimensions and complex maps in higher dimensions.

TABLE XI
EXPERIMENTAL RESULTS OF FOUR ALGORITHMS IN A 30*30 RASTER MAP ENVIRONMENT

Algorithm name	Distance calculation	$\eta(t)$	$d(t)$	$e(t)$
a)	Euclidean distance	779	769	14096.71
b)	Euclidean distance	732	722	14526.66
c)	Chebyshev distance	725	715	14712.74
c)	Manhattan distance	729	719	14549.21
d)	Chebyshev distance	707	697	14849.47
d)	Manhattan distance	713	703	14743.75

In the 10*10 and 20*20 raster maps, the IQL algorithm using Chebyshev distance significantly reduces the trials needed for convergence—by 15.29% and 22.63%, respectively—while also increasing the expected return by 7.23% and 9.11%. The algorithm performs even better in the 20*20 environment, demonstrating its effectiveness in handling larger, more complex problems. In the 30*30 map, although improvements are smaller, the algorithm still reduces trials by 9.24% and increases expected returns by 5.34%. Table XII shows the optimized algorithm's performance compared to the original algorithm.

TABLE XII
PERFORMANCE COMPARISON BETWEEN THE OPTIMIZED ALGORITHM AND THE ORIGINAL ALGORITHM

Algorithm	Distance	Evaluation	10*10	20*20	30*30
d)	Chebyshev	$\eta(t)$	15.29%	22.63%	9.24%
d)	Chebyshev	$d(t)$	16.33%	23.11%	9.36%
d)	Chebyshev	$e(t)$	7.23%	9.11%	5.34%
d)	Manhattan	$\eta(t)$	9.55%	16.05%	8.47%
d)	Manhattan	$d(t)$	10.21%	16.39%	8.58%
d)	Manhattan	$e(t)$	5.63%	6.84%	4.59%

Chebyshev distance is chosen over Manhattan distance for its more accurate modeling of movement costs in grid-based environments, where diagonal movements are equally important as horizontal or vertical ones. This choice enhances the efficiency and accuracy of the IQL algorithm across various map dimensions, with observed performance improvements further validating its effectiveness.

C. Algorithm comparison

This section highlights the advantages of the IQL algorithm by comparing it with the FIQL, PP-Q-Learning-based CPP (PP-QL-based CPP), DFQL, and QMABC algorithms. The comparison follows these steps.

Step1: Clarify the purpose of the comparison.

This program aims to compare the performance of different algorithms in 10*10 and 20*20 raster environments. By examining these scenarios, we can understand the variation in efficiency and effectiveness of three algorithms across different raster map sizes.

Step2: Selection of benchmarking algorithms.

This section compares the IQL algorithm with the FIQL, PP-QL-based CPP, DFQL, and QMABC algorithms. These comparison algorithms are all variants of Q-learning that use different approaches to enhance learning speed and performance. This comparison demonstrates the superiority of the IQL algorithm.

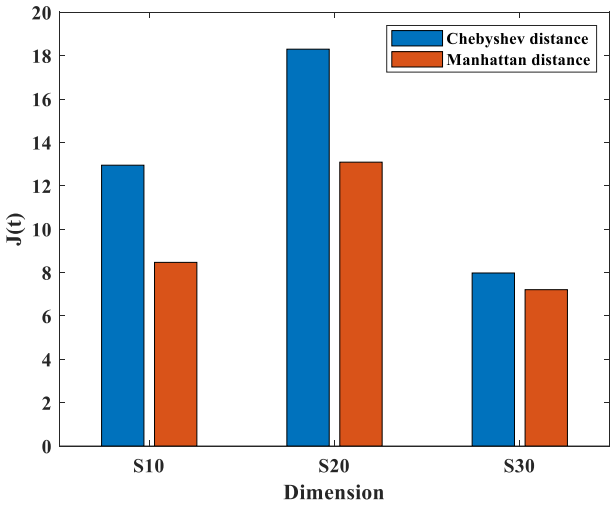

Fig. 12. Comparison of Algorithm Performance Using Different Distance Calculation Methods

Step3: Algorithm performance evaluation metrics.

Evaluation metrics are crucial for measuring algorithm performance. We will use three metrics from Section Methods: the number of trials X under consistent iterations ($\eta(t)$), the number of trials where the standard deviation reaches 0 ($d(t)$), and the expected return after convergence ($e(t)$). All algorithms must be run under the same conditions and tested on the same dataset. Additionally, the performance of each metric should be recorded and statistically analyzed to identify the best-performing algorithm for the given scenario.

Step4: Comparative analysis of algorithms.

Table XIII and Table XIV compared the experimental results of the Q-learning algorithm with the IQL algorithm, the FIQL algorithm, the PP-QL-based CPP algorithm, the DFQL algorithm, and the QMABC algorithm.

TABLE XIII
THE EXPERIMENTAL RESULTS OF THE THREE ALGORITHMS ARE SHOWN AND COMPARED (10*10)

Algorithm name	$\eta(t)$	$d(t)$	$e(t)$
a). Q-learning algorithm	165	155	2543.26
d). IQL algorithm	133	123	2726.95
FIQL algorithm	144	134	2688.18
PP-QL-based CPP algorithm	141	131	2694.95
DFQL algorithm	149	139	2597.78
QMABC algorithm	146	136	2622.12

TABLE XIV
THE EXPERIMENTAL RESULTS OF THE THREE ALGORITHMS ARE SHOWN AND COMPARED (20*20)

Algorithm name	$\eta(t)$	$d(t)$	$e(t)$
a) Q-learning algorithm	486	476	8794.61
d) IQL algorithm	376	366	9594.86
FIQL algorithm	406	396	9297.78
PP-QL-based CPP algorithm	392	382	9322.12
DFQL algorithm	427	417	9367.78
QMABC algorithm	415	405	9372.12

In the 10*10 and 20*20 raster map environments, the IQL algorithm demonstrated significant performance advantages. For example, in the 10*10 grid, compared to traditional Q-learning, the trials required to reach a steady state with a standard deviation of 0 were reduced by 20.65%, 13.55%, 15.48%, 10.32%, and 12.26% for the IQL, FIQL, PP-QL-based CPP, DFQL, and QMABC algorithms, respectively. In the 20*20 grid, the reductions were 20.11%, 16.81%, 19.75%, 12.39%, and 14.92%, respectively. Among the five algorithms, IQL exhibited the fastest convergence speed and the highest stability, enabling more effective task learning and execution, and achieving optimal performance.

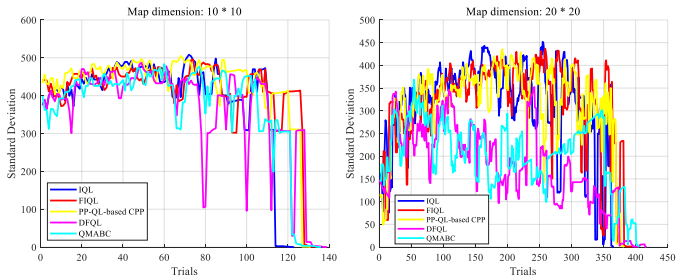

Fig. 13. Comparison of the number of trials in which the standard deviation of the five different algorithms reaches 0 for two types of raster environments

The graphs illustrate the results of experiments in two different raster environments, comparing the FIQL algorithm, the PP-QL-based CPP algorithm, and the improved IQL algorithm. This section focuses on the number of trials where the standard deviation reaches 0, a key stability and convergence speed metric. A smaller standard deviation indicates more stable data; a standard deviation 0 means complete stability. As shown in Fig. 13, the IQL algorithm converges faster than the other two algorithms in both raster environments. This quick convergence is valuable for practical applications as it reduces the time needed for learning and planning. The IQL algorithm not only converges faster but also achieves better final performance.

The IQL algorithm enhances learning efficiency and task performance within the same environment, boosting overall performance, which is crucial for tackling more complex and larger-scale problems. In summary, its performance in a raster environment demonstrates its effectiveness in improving learning efficiency, stability, and outcomes, offering valuable insights for future algorithm design and optimization in similar settings.

D. Discussion

Our work presents an optimized algorithm that enhances Q-table initialization and refines reward functions, enabling faster and more efficient strategy development in complex environments by minimizing unnecessary trial and error. This approach shows broad applicability across various raster map environments, consistently improving the IQL algorithm's ability to navigate high-dimensional maps with numerous obstacles. Experimental results confirm the algorithm's stability and reliability, validating its effectiveness in path planning through both theoretical and empirical analysis.

The IQL algorithm shows enhanced performance across all map dimensions. In low-dimensional maps, it efficiently

guides the agent to avoid obstacles and find the optimal path, significantly outperforming traditional Q-learning. Even in more complex 30*30 maps, the algorithm reduces trials needed for convergence and increases expected returns, consistently providing better stability, faster convergence, and more effective pathfinding across all tested dimensions.

These findings have significant implications for practical applications, particularly in the operation of autonomous robots or vehicles in complex and dynamic environments. Although the IQL algorithm demonstrates superior stability and convergence speed under certain conditions, further optimization is needed to enhance its adaptability and decision-making speed in highly complex, dynamic environments. Future research should focus on improving the algorithm's robustness in such environments to ensure that autonomous systems can achieve efficient and stable path planning across various real-world scenarios. This will not only increase the practical value of the algorithm but also advance the development of autonomous robotics and vehicle technologies.

V. Conclusion

We present an IQL algorithm that leverages the PACO algorithm to optimize initial Q-table values, enabling the agent to quickly adapt and select optimal paths. This approach addresses key challenges in path planning within complex environments, including slow convergence, local optimization issues, and the need for precise environment modeling. Furthermore, we incorporated a UCH reward function mechanism to alleviate reward function sparsity, reduce exploration difficulties, and enhance reward evaluation. Compared to traditional methods, the IQL algorithm significantly improves performance, speed, and accuracy, effectively meeting the demands of path planning and obstacle avoidance. Our results indicate that the model demonstrates strong generalization capabilities across various scenarios.

References

[1] M. B. Alatise and G. P. Hancke, "A Review on Challenges of Autonomous Mobile Robot and Sensor Fusion Methods," in *IEEE Access*, vol. 8, pp. 39830–39846, Feb. 2020.

[2] F. Rubio, F. Valero and C. Llopis-Albert, "A review of mobile robots: Concepts, methods, theoretical framework, and applications," in *International Journal of Advanced Robotic Systems*, vol. 16, no. 2, 2019.

[3] X. S. Xiao, B. Liu, G. Warnell *et al*., "Motion planning and control for mobile robot navigation using machine learning: a survey," *Auton Robot*, vol. 46, pp. 569–597, Jun. 2022.

[4] S. Y. Teng *et al*., "Motion Planning for Autonomous Driving: The State of the Art and Future Perspectives," in *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 6, pp. 3692-3711, Jun. 2023.

[5] Z. Z. Chu, F. L. Wang and T. J. Lei, "Path Planning Based on Deep Reinforcement Learning for Autonomous Underwater Vehicles Under Ocean Current Disturbance," in *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 1, pp. 108-120, Jan. 2023.

[6] Y. H. Sun, M. Fang and Y. X. Su, "AGV Path Planning based on Improved Dijkstra Algorithm," in *Journal of Physics: Conference Series*, vol. 1746, pp. 22-23, Nov. 2020, Beijing, China.

[7] M. Parimala, S. Broumi, K. Prakash *et al*., "Bellman–Ford algorithm for solving shortest path problem of a network under picture fuzzy environment," in *Complex Intelligent Systems*, vol 7, pp. 2373–2381, Jun 2021.

[8] G. Tang *et al*., "Geometric A-Star Algorithm: An Improved A-Star Algorithm for AGV Path Planning in a Port Environment," in *IEEE Access*, vol. 9, pp. 59196-59210, 2021.

[9] J. K. Wang *et al*., "Neural RRT*: Learning-Based Optimal Path Planning," in *IEEE Transactions on Automation Science and Engineering*, vol. 17, no. 4, pp. 1748-1758, Oct. 2020.

[10] Y. L. Chen *et al*., "Path Planning and Obstacle Avoiding of the USV Based on Improved ACO-APF Hybrid Algorithm With Adaptive Early-Warning," in *IEEE Access*, vol. 9, pp. 40728-40742, 2021.

[11] B. Y. Song, Z. D. Wang and L. Zou, "An improved PSO algorithm for smooth path planning of mobile robots using continuous high-degree Bezier curve," in *Applied Soft Computing*, vol 100, Mar. 2021, Art. no. 106960.

[12] Y. B. Li, D. G. Dong and X. N. Guo, "Mobile Robot Path Planning based on Improved Genetic Algorithm With A-star Heuristic Method," in 2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference (ITAIC), Chongqing, China, 2020, pp. 1306-1311.

[13] B. Y. Wang, Z. Liu and Q. B. Li, "Mobile Robot Path Planning in Dynamic Environments Through Globally Guided Reinforcement Learning," in *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6932-6939, Oct. 2020.

[14] Z. X. Liu *et al*., "MAPPER: Multi-Agent Path Planning with Evolutionary Reinforcement Learning in Mixed Dynamic Environments," in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 2020, pp. 11748-11754.

[15] L. He, N. Aouf and B. F. Song, "Explainable Deep Reinforcement Learning for UAV autonomous path planning," in *Aerospace Science and Technology*, vol. 118, Nov. 2021, Art. no. 107052.

[16] M. Theile *et al*., "UAV Path Planning using Global and Local Map Information with Deep Reinforcement Learning," 2021 20th International Conference on Advanced Robotics (ICAR), Ljubljana, Slovenia, 2021, pp. 539-546, doi: 10.1109/ICAR53236.2021.9659413.

[17] P. Z. Chen, J. A. Pei, W. Q. Lu and M. Z. Li, "A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance," in *Neurocomputing*, vol. 497, Aug. 2022, pp. 64-75.

[18] Q. Liu *et al*., "Path Planning for UAV-Mounted Mobile Edge Computing With Deep Reinforcement Learning," in *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5723-5728, May 2020.

[19] Y. H. Cho, D. Baek, J. Kim and N. Chang, "Energy-Efficient Online Path Planning of Multiple Drones Using Reinforcement Learning," in *IEEE Transactions on Vehicular Technology*, vol. 70, no. 10, pp. 9725-9740, Oct. 2021.

[20] L. Y. Li, D. F. Wu, Y. Q. Huang and Z. M. Yuan, "A path planning strategy unified with a COLREGS collision avoidance function based on deep reinforcement learning and artificial potential field," in *Applied Ocean Research*, vol 113, Aug. 2021, Art. no. 102759.

[21] X. L. Xu *et al*., "Path planning and dynamic collision avoidance algorithm under COLREGs via deep reinforcement learning," in *Neurocomputing*, vol 468, pp. 181-197, Jan 2022.

[22] H. T. Hu, X. R. Yang, S. C. Xiao and F. Y. Wang, "Anti-Conflict AGV Path Planning in Automated Container Terminals Based on Multi-Agent Reinforcement Learning," in *International Journal of Production Research*, vol 61, no. 1 , pp. 65–80, 2023.

[23] S. Josef and A. Degani, "Deep Reinforcement Learning for Safe Local Planning of a Ground Vehicle in Unknown Rough Terrain," in *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6748-6755, Oct. 2020.

[24] B. Hao, H. Du and Z. P. Yan, "A path planning approach for unmanned surface vehicles based on dynamic and fast Q-learning," in *Ocean Engineering*, vol. 270, Feb. 2023, Art. no. 113632.

[25] A. Maoudj and A. Hentout, "Optimal path planning approach based on Q-learning algorithm for mobile robots," in *Applied Soft Computing*, vol. 97, Part A, Dec. 2020, Art. no. 106796.

[26] E. S. Low, P. Ong and C. K. Chun, "Solving the optimal path planning of a mobile robot using improved Q-learning," in *Robotics and Autonomous Systems*, vol. 115, pp. 143-161, May 2019.

[27] B. Hao *et al*., "A search and rescue robot search method based on flower pollination algorithm and Q-learning fusion algorithm," in *PLOS ONE*, vol. 18, no. 3, e0283751, Mar. 2023, https://doi.org/10.1371/journal.pone.0283751

[28] X. R. Ni *et al*., "A Q-learning based multi-strategy integrated artificial bee colony algorithm with application in unmanned vehicle path planning," in *Expert Systems with Applications*, vol. 236, Feb. 2024, Art. no. 121303.

[29] P. K. Das, H. S. Behera and B. K. Panigrahi, "Intelligent-based multi-robot path planning inspired by improved classical Q-learning and improved particle swarm optimization with perturbed velocity," in *Engineering Science and Technology, an International Journal*, vol. 19, Issue 1, Mar. 2016, pp. 651-669.

[30] E. S. Low, P. Ong and C. Y. Low, "A modified Q-learning path planning approach using distortion concept and optimization in dynamic environment for autonomous mobile robot," in *Computers Industrial Engineering*, vol. 181, Jul. 2023, Art. no. 109338.
[31] A. Puente-Castro *et al*., "Enrique Fernandez-Blanco,Q-Learning based system for Path Planning with Unmanned Aerial Vehicles swarms in obstacle environments," in *Expert Systems with Applications*, vol. 235, Jan. 2024, Art. no. 121240.
[32] X. B. Yu and W. G. Luo, "Reinforcement learning-based multi-strategy cuckoo search algorithm for 3D UAV path planning," in *Expert Systems with Applications*, vol. 223, Aug. 2023, Art. no. 119910.
[33] G. Kulathunga, "A Reinforcement Learning based Path Planning Approach in 3D Environment," in *Procedia Computer Science*, vol. 212, pp. 152-160, 2022.
[34] G. T. Lee, K. J. Kim and J. Jang, "Real-time path planning of controllable UAV by subgoals using goal-conditioned reinforcement learning," in *Applied Soft Computing*, vol. 146, Oct. 2023, Art. no. 110660.
[35] J. Petrik and M. Bambach, "Reinforcement learning and optimization based path planning for thin-walled structures in wire arc additive manufacturing," in *Journal of Manufacturing Processes*, vol. 93, pp. 75-89, May 2023.
[36] M. Zhao *et al*., "A fast robot path planning algorithm based on bidirectional associative learning," in *Computers Industrial Engineering*, vol. 155, May 2021, Art. no. 107173.
[37] P. Ladosz, L. L. Weng, M. Kim and H. Oh, "Exploration in deep reinforcement learning: A survey," in *Information Fusion*, vol. 85, pp. 1-22, Sep. 2022.
[38] M. Y. Zhang, W. Y. Cai and L. F. Pang, "Predator-Prey Reward Based Q-Learning Coverage Path Planning for Mobile Robot," in *IEEE Access*, vol. 11, pp. 29673-29683, 2023.
[39] Z. H. Ding, Y. H. Huang, H. Yuan and H. Dong, "Introduction to Reinforcement Learning," in *Deep Reinforcement Learning*, Springer, Singapore, 2020.
[40] W. Li *et al*., "Secure and Reliable Downlink Transmission for Energy-Efficient User-Centric Ultra-Dense Networks: An Accelerated DRL Approach," in *IEEE Transactions on Vehicular Technology*, vol. 70, no. 9, pp. 8978-8992, Sep. 2021.
[41] T. Tan, H. Xie and L. Feng, "Q-learning with heterogeneous update strategy," in *Information Sciences*, vol. 656, Jan. 2024, Art. no. 119902.
[42] T. G. Karimpanal *et al*., "Balanced Q-learning: Combining the influence of optimistic and pessimistic targets," in *Artificial Intelligence*, vol. 325, Dec. 2023, Art. no. 104021.
[43] X. Y. Li *et al*., "Deep reinforcement learning for optimal rescue path planning in uncertain and complex urban pluvial flood scenarios," in *Applied Soft Computing*, vol. 144, Sep. 2023, Art. no. 110543.
[44] E. S. Low, P. Ong and C. Y. Low, "An empirical evaluation of Q-learning in autonomous mobile robots in static and dynamic environments using simulation," in *Decision Analytics Journal*, vol. 8, Sep. 2023, Art. no. 100314.
[45] M. García, N. López and I. Rodríguez, "A full process algebraic representation of Ant Colony Optimization," in *Information Sciences*, vol. 658, Feb. 2024, Art. no. 120025.
[46] M. Tadaros and N. A. Kyriakakis, "A Hybrid Clustered Ant Colony Optimization Approach for the Hierarchical Multi-Switch Multi-Echelon Vehicle Routing Problem with Service Times," in *Computers Industrial Engineering*, vol. 190, Apr. 2024, Art. no. 110040.
[47] T. L. Li *et al*., "Gaussian distribution resampling via Chebyshev distance for food computing," in *Applied Soft Computing*, vol. 150, Jan. 2024, Art. no. 111103.
[48] C. D. Wang, J. L. Yang and B. Q. Zhang, "A fault diagnosis method using improved prototypical network and weighting similarity-Manhattan distance with insufficient noisy data," in *Measurement*, vol. 226, Feb. 2024, Art. no. 114171.

Wei Liu received a B.S. in Mathematics Education from Shenyang Normal University in 2000, an M.S. in Computer Application Technology from Liaoning Technical University in 2008, and a Ph.D. in Mining Engineering from Liaoning Technical University in 2019. He is currently an Associate Professor at the College of Science, Liaoning Technical University, and the Deputy Director of the Institute of Mathematics and System Science. He is a member of several professional societies, including the Mathematical Planning Branch of the Chinese Operations Research Association, the Chinese Computer Society, the Chinese Artificial Intelligence Society, and the Chinese Coal Association. His research interests encompass machine learning, deep neural networks, and unmanned truck intelligent dispatching mining system engineering. He has published over 30 academic articles in core journals such as the Journal of Computer Science and Control and Decision-Making. He has participated in seven National Natural Science Foundation of China projects, eight provincial and ministerial-level scientific research projects, and has led four enterprise projects. Additionally, he has contributed to over 20 enterprise projects and has received multiple awards, including two second-class awards, one third-class award, and one second-class award from the China Coal Industry Science and Technology Awards.

Ruiyang Wang received a B.S. degree in information and computing science from Liaoning Technical University, Liaoning, China, in 2023. She is currently working toward an M.S. degree in mathematics major with Liaoning Technical University, Liaoning, China. Her research interests include path planning, deep learning, neural networks, and deep reinforcement learning.

Haonan Wang received a B.S. degree in information and computing Science from Liaoning Technical University, Fuxin, Liaoning, China. He is currently studying M.S. degree in Computer Science at Johns Hopkins University, Maryland, USA. His research interests are the intersection of Human-computer interaction and Machine Learning/ Natural Language Processing, especially Large Language Models and Agents for Human-centered evaluation in interaction systems.

Guangwei Liu is now a professor at the College of Mines, Liaoning Technical University, Liaoning, China. He is a part-time professor at the National Engineering Research Center for Computer Software. He was awarded the National Coal Youth Science and Technology Prize, and May 4th Youth Medal, and selected as one of the "One Hundred Million Talents" projects in Liaoning Province. His research interests include intelligent mining in open pit mines, system optimization, and digital mine technology. He has presided over many national and provincial key projects and won 4 first prizes and 6-second prizes of provincial and ministerial level science and technology awards. He has published more than 30 papers, 4 authorized patents, and 12 software copyrights.